Smart Agriculture System (AgriBot)

An IoT-Enabled Autonomous Agriculture Robot

Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science and Engineering of the University of Asia Pacific

by

Jubayer Hasan Samir
Roll: 22201153

Nazmul Mridha
Roll: 22201161

Arman Alvi
Roll: 22201164

Supervised by:

Nazmun Nahid
Assistant Professor
Department of Computer Science and Engineering
University of Asia Pacific

Department of Computer Science & Engineering
University of Asia Pacific

May 2026

Declaration

We, hereby, declare that the work presented in this report is the outcome of the investigation performed by us under the supervision of Nazmun Nahid, Assistant Professor, Department of Computer Science and Engineering, University of Asia Pacific. We also declare that no part of this report and thereof has been or is being submitted elsewhere for the award of any degree or diploma.

Countersigned

(Nazmun Nahid)
Supervisor

Signature

.....
Jubayer Hasan Samir

.....
Nazmul Mridha

.....
Arman Alvi

Acknowledgements

First of all, we would like to thank Almighty Allah for giving us the strength, ability, and patience to complete this project successfully.

We would like to express our sincere gratitude to our respected supervisor, **Nazmun Nahid**, Assistant Professor, Department of Computer Science and Engineering, University of Asia Pacific. Her guidance, encouragement, and valuable feedback were essential in shaping both the technical and academic aspects of this project. Despite her busy schedule, she always found time to help us through challenges.

We are also grateful to the Department of Computer Science and Engineering, University of Asia Pacific, for providing us with the necessary resources and laboratory support throughout this project.

Finally, we thank our families and friends for their continuous moral support and motivation throughout this journey.

Abstract

Agriculture is the backbone of Bangladesh's economy, yet it continues to rely heavily on manual labour, making it slow, costly, and inefficient. This project presents AgriBot, an IoT-enabled autonomous agriculture robot designed to address the core challenges of modern farming. Built on the ESP32 DevKit V1 microcontroller, AgriBot integrates two IR sensors for autonomous navigation, an HC-SR04 ultrasonic sensor for real-time obstacle detection, a single L298N motor driver module for left and right DC motor control, two relay modules for crops cutting mechanism and water pump/sprayer switching, and environment sensors including a DHT22 temperature and humidity sensor, a rain sensor, and a soil moisture sensor for smart irrigation decisions. The robot operates in both autonomous and manual modes, with a WiFi-based web dashboard accessible at 192.168.4.1 allowing real-time monitoring and remote control from any smartphone or browser. Sensor data including temperature, humidity, soil moisture, rain level, and obstacle distance are displayed live on a 16x2 I2C LCD display. AgriBot demonstrates the feasibility of low-cost, scalable agricultural automation for small and medium farms, reducing human effort and improving crop management efficiency.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Agriculture contributes approximately 13% to Bangladesh's GDP and employs nearly 40% of its labour force, yet it still relies heavily on manual methods for crop monitoring, irrigation, and harvesting — processes that are slow, labour-intensive, and prone to water wastage and crop damage.

The growth of IoT and embedded systems has enabled smart robots that monitor soil conditions, detect weather changes, navigate fields autonomously, and execute farming tasks with precision, reducing human effort and improving productivity.

Motivated by this, this project proposes AgriBot — an IoT-enabled autonomous agriculture robot for paddy field operations. AgriBot navigates crop rows using IR sensors, detects and avoids obstacles, activates a crops cutting mechanism and water sprayer via relays, and continuously monitors field conditions, all while being monitored remotely through a WiFi web dashboard.

1.2 Problem Statement

Traditional farming in Bangladesh faces several challenges: manual crop monitoring and harvesting are time-consuming and labour-intensive; irrigation decisions are made without real-time soil moisture or rainfall data, causing water wastage; farmers lack an easy way to remotely monitor field conditions; field obstacles can damage manual equipment; and the lack of automation increases operational cost and dependency on seasonal labour. These gaps highlight the need for an intelligent, sensor-equipped robotic system capable of operating autonomously in agricultural environments.

1.3 Objectives of the Project

The key objectives of AgriBot are as follows:

1. Design and build an autonomous agriculture robot using the ESP32 DevKit V1 microcontroller.
2. Implement a two-sensor IR-based auto navigation system for crop row following.
3. Integrate an HC-SR04 ultrasonic sensor for real-time obstacle detection and avoidance.
4. Develop a relay-controlled crops cutting mechanism and water sprayer for agricultural actuation.
5. Build a smart monitoring system that reads soil moisture, rain level, temperature, and humidity in real time.
6. Create a WiFi-based web dashboard hosted at 192.168.4.1 for real-time monitoring and remote manual control.
7. Display live sensor data on a 16x2 I2C LCD display.

1.4 Research Questions

1. Can an ESP32-based robot reliably navigate paddy crop rows using a two-IR-sensor array in automatic mode?
2. Can real-time obstacle detection using the HC-SR04 sensor prevent collisions during autonomous operation?
3. Can sensor-based monitoring (soil moisture, rain, DHT22) provide farmers with sufficient data to make informed irrigation decisions?
4. Is a WiFi-hosted web dashboard sufficient for real-time remote monitoring and control of an agriculture robot?

1.5 Scope of the Work

This project covers hardware design and wiring of all robot components (motors, sensors, relays, LCD), firmware development in C++ using the Arduino framework for ESP32, implementation of IR-based navigation, ultrasonic obstacle avoidance, relay actuator control and sensor monitoring logic, a WiFi-based web dashboard with

WebSocket communication, and testing of individual modules and integrated performance. The project does not cover GPS-based navigation, computer vision, machine learning, or large-scale industrial deployment.

1.6 Expected Outcomes

By the end of this project, the following outcomes are expected: a fully functional AgriBot capable of autonomous paddy field operation; reliable two-sensor IR-based navigation with obstacle detection stopping the robot at 15 cm; relay-controlled crops cutting mechanism and water sprayer operable from the dashboard; a live WiFi dashboard displaying all sensor readings with full remote control; and a 16x2 LCD cycling through temperature, humidity, rain, object distance, soil moisture, mode, and relay status.

1.7 Impacts of the Project

Societal and Cultural Impact: AgriBot reduces dependency on seasonal manual labour, enabling farmers — including elderly or physically limited individuals — to manage their fields more independently.

Health, Safety, and Legal Impact: Automating crop cutting and field monitoring reduces farmers' exposure to hazardous conditions such as heat, sharp tools, and pesticides. The obstacle detection system prevents unintended collisions, and the active-LOW relay logic acts as a fail-safe measure.

Environmental and Sustainability Impact: Soil moisture and rain sensor readings enable informed water management, reducing wasteful irrigation. The robot runs on a 12V battery with low-power sensors, minimising energy consumption and preserving soil health.

1.8 Report Organization

The remainder of this report is organised as follows. Chapter 2 presents the related works review of existing agriculture robotics systems. Chapter 3 describes the robot structure details and hardware setup including wiring and GPIO assignments. Chapter 4 presents the algorithm design and pseudocode. Chapter 5 provides a discussion of results and evaluation. Chapter 6 concludes the report with a summary and future work directions.

Chapter 2

Related Works

This chapter provides the theoretical foundation and a review of related research work relevant to AgriBot. It begins with essential concepts in the Preliminaries section, followed by a detailed survey of existing systems, techniques, and studies in the literature.

2.1 Preliminaries

Internet of Things (IoT)

IoT refers to the network of physical devices embedded with sensors, software, and communication technologies that enable data collection and exchange over the internet. In agriculture, IoT devices are used for soil monitoring, weather sensing, and automated control of irrigation and machinery [3].

ESP32 DevKit V1 Microcontroller

The ESP32 DevKit V1 is a dual-core, 32-bit microcontroller with integrated WiFi and Bluetooth. It supports hardware PWM, I2C communication, multiple ADC channels (12-bit resolution), and more than 30 GPIO pins. For AgriBot, the ESP32 operates as a WiFi Access Point (SSID: *Smart Agriculture Robot*, IP: 192.168.4.1) hosting both an HTTP web server and a WebSocket server for real-time bidirectional communication.

IR-Based Auto Navigation

Infrared (IR) sensors detect the contrast between surfaces by emitting an IR beam and reading the reflected signal. For AgriBot, a left IR sensor (GPIO 32) and a right IR sensor (GPIO 33) detect the presence of a crop row line. The auto-navigation logic derives direction from the combined state of both sensors: both detected means

move forward; only right detected means turn right; only left detected means turn left; neither detected means stop [2].

Ultrasonic Distance Sensing

The HC-SR04 ultrasonic sensor measures distance by emitting a sound pulse from the TRIG pin and measuring the time taken for the echo to return to the ECHO pin. Distance is calculated as:

$$d = \frac{t \times 0.0343}{2}$$

where t is the echo pulse duration in microseconds and the result d is in centimetres. The ECHO pin outputs a 5 V signal, so a voltage divider (1 kΩ and 2 kΩ resistors) is used to bring it to approximately 3.3 V before connecting to ESP32 GPIO 23.

DHT22 Temperature and Humidity Sensor

The DHT22 provides calibrated digital temperature and humidity readings over a single-wire protocol. It has higher accuracy and wider range than the DHT11, with a sampling rate limited to one reading every 2.5 seconds. A 10 kΩ pull-up resistor on the DATA line (GPIO 27) is required for reliable communication.

Relay Module Operation

A relay is an electrically controlled switch. The relay modules used in AgriBot are active-LOW devices: a LOW signal from the ESP32 GPIO activates the relay, connecting the load circuit. Relay 1 (GPIO 25) controls the Crops Cutting Mechanism and Relay 2 (GPIO 26) controls the Water Pump/Sprayer.

I2C Communication and LCD Display

The 16x2 LCD display uses the I2C protocol through a PCF8574T I/O expander at address 0x27. For AgriBot, the default ESP32 I2C pins (GPIO 21 and GPIO 22) are already occupied by the motor driver, so a custom I2C bus is initialised on GPIO 16 (SDA) and GPIO 17 (SCL) using `Wire.begin(16, 17)`.

Motor Driver — L298N Module

The L298N is a dual H-bridge motor driver capable of controlling two DC motors independently. For AgriBot, IN1 and IN2 (GPIO 18 and GPIO 19) control the Left Motor, while IN3 and IN4 (GPIO 21 and GPIO 22) control the Right Motor. The

motor power input is connected to an external 12 V battery, keeping motor current completely separate from the ESP32.

2.2 Related Works / Existing Systems

Several agriculture robot systems have been proposed in recent literature:

Automated Irrigation Robot (Kumar et al., 2021) [4] proposed a soil-moisture-based irrigation robot using Arduino Uno. The system used a single soil sensor and a DC pump, but lacked navigation, obstacle avoidance, or remote monitoring capabilities.

Rice Field Line-Following Robot (Hossain et al., 2020) [1] implemented a 3-sensor IR array on a differential drive robot for paddy field navigation. While line-following was achieved, the system had no crop cutting, irrigation, or IoT connectivity.

IoT-Based Smart Farm Monitoring (Patil et al., 2022) [5] developed a stationary IoT system using ESP8266 for soil, temperature, and humidity monitoring with a mobile app. The system provided good monitoring capability but had no autonomous robotic mobility or actuation.

Autonomous Paddy Harvesting Robot (Zhang et al., 2019) [6] demonstrated a GPS-guided harvesting robot for large paddy fields. While the system was highly capable, it required expensive GPS hardware and was not affordable for small farmers.

2.3 Comparative Analysis of Related Works

Table 2.1 summarises the key features of related systems compared to AgriBot.

Table 2.1: Comparative Analysis of Related Agriculture Robot Systems

System	**Navigation**	**Obstacle Avoid**	**Env. Monitor**	**Crop Cut**	**WiFi Dashboard**
Kumar et al. (2021)	No	No	Soil only	No	No
Hossain et al. (2020)	IR (3)	No	No	No	No
Patil et al. (2022)	No	No	Yes	No	Yes
Zhang et al. (2019)	GPS	Yes	No	Yes	No
AgriBot (Ours)	**IR (2)**	**Yes**	**Yes (4 sensors)**	**Yes**	**Yes**

2.4 Research Gap / Limitations of Existing Methods

From the comparative analysis, the following gaps are identified in existing systems:

- No existing system combines IR-based navigation, obstacle avoidance, crop cutting, full environmental monitoring, and WiFi remote control in a single affordable platform.
- Most systems use expensive GPS hardware or are stationary monitoring nodes without mobility.
- None of the reviewed systems combine soil moisture, rain, temperature, and humidity monitoring together with actuator control from a real-time web dashboard.
- None provide a WebSocket-based real-time dashboard for both manual and autonomous control.

AgriBot addresses all these gaps by integrating all features on the affordable ESP32 DevKit V1 platform.

2.5 Summary

This chapter reviewed the theoretical foundations of IoT, IR navigation, ultrasonic sensing, motor control, and relay actuation relevant to AgriBot. A survey of existing agriculture robot systems revealed that no single affordable system combines autonomous navigation, crop cutting, smart irrigation, and WiFi monitoring. AgriBot is designed to fill this gap with a low-cost, fully integrated solution.

Chapter 3

Robot Structure Details / Hardware Setup

This chapter describes the complete robot structure, hardware components, circuit wiring, and GPIO pin assignments of AgriBot.

3.1 System Overview

AgriBot is a differential-drive wheeled robot built on the ESP32 DevKit V1 microcontroller. As shown in the Embedded System Block Diagram (Figure 3.1), the system is organised into four main blocks:

1. **Input Blocks (Sensing Layer):** Six sensors feed data into the ESP32 — Soil Moisture Sensor, Rain Sensor, DHT22 Temperature & Humidity Sensor, Left IR Sensor, Right IR Sensor, and HC-SR04 Ultrasonic Sensor.
2. **Processing Block (ESP32 DevKit Controller):** The ESP32 firmware executes Manual Mode and Auto Navigation Logic, Obstacle Detection, Soil/Rain/DHT22 monitoring, WiFi Access Point management, and Web Dashboard Server hosting.
3. **Output Blocks (Actuation Layer):** The L298N Motor Driver Module drives the Left DC Motor and Right DC Motor. Relay 1 activates the Crops Cutting System. Relay 2 activates the Water Spray System. The 16x2 I2C LCD Display shows live sensor status.
4. **Communication Block:** The ESP32 creates a WiFi Access Point with SSID *Smart Agriculture Robot*. The dashboard is accessible at IP address 192.168.4.1 from any connected smartphone or browser.

3.1.1 Embedded System Block Diagram

Figure 3.1 shows the complete embedded system block diagram of AgriBot, illustrating the relationship between the input sensor blocks, the ESP32 DevKit Controller processing block, the WiFi communication block, and the output actuator blocks.

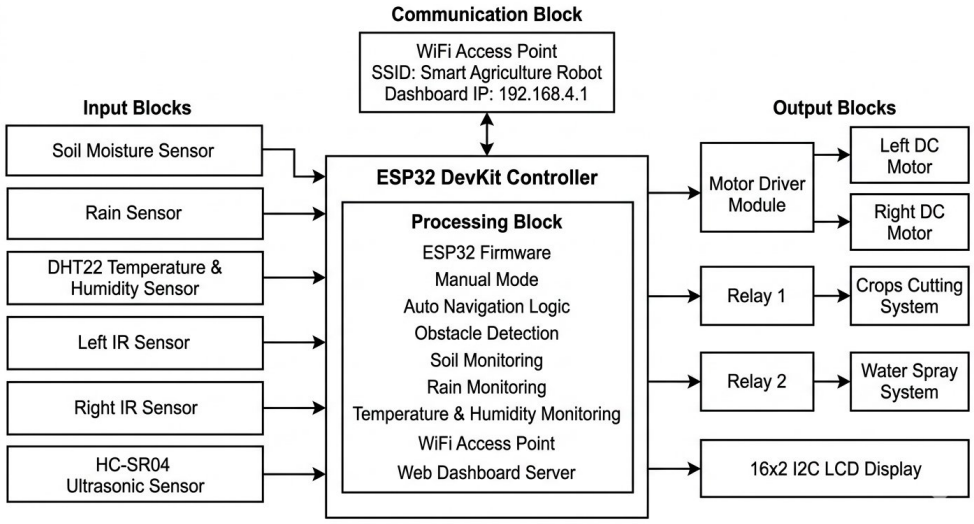


Figure 3.1: AgriBot Embedded System Block Diagram

As shown in Figure 3.1, six sensors form the Input Blocks feeding data into the ESP32 DevKit Controller, which runs firmware for Manual Mode, Auto Navigation, Obstacle Detection, environmental monitoring, WiFi AP management, and dashboard hosting. The Output Blocks consist of the Motor Driver (Left/Right DC Motors), Relay 1 (Crops Cutting), Relay 2 (Water Spray), and the 16x2 I2C LCD. The Communication Block shows the WiFi AP (SSID: Smart Agriculture Robot, IP: 192.168.4.1) providing bidirectional communication with connected devices.

3.2 System Components

Table 3.1 lists all major components used in AgriBot.

Table 3.1: AgriBot System Components and Technologies

Component	**Model / Type**	**Function**
Microcontroller	ESP32 DevKit V1	Main processing, WiFi AP, PWM
Motor Driver	L298N Module	Controls Left and Right DC Motors
Relay Module 1	Active-LOW 5 V Relay	Crops Cutting Mechanism ON/OFF
Relay Module 2	Active-LOW 5 V Relay	Water Pump/Sprayer ON/OFF
Left IR Sensor	Digital IR module	Auto navigation (left side)
Right IR Sensor	Digital IR module	Auto navigation (right side)
Ultrasonic Sensor	HC-SR04	Obstacle detection
Temperature/Humidity	DHT22 module	Environment sensing
Rain Sensor	Analog module (AO)	Rain level detection
Soil Moisture Sensor	Analog module (AO)	Soil wetness detection
LCD Display	16x2 I2C (PCF8574T)	Live status display (addr. 0x27)
Resistor	1 kΩ (R2)	Ultrasonic ECHO voltage divider
Resistor	2 kΩ (R3)	Ultrasonic ECHO voltage divider
Resistor	10 kΩ (R1)	DHT22 DATA pull-up
Battery	12 V DC	Motor driver power supply

3.3 GPIO Pin Assignment

Table 3.2 shows the complete GPIO pin assignment for AgriBot as implemented in the firmware.

Table 3.2: ESP32 DevKit V1 — Complete GPIO Pin Assignment

Module / Component	**Module Pin**	**ESP32 GPIO**	**Notes**
L298N Motor Driver	IN1	GPIO 18	Left Motor direction A
L298N Motor Driver	IN2	GPIO 19	Left Motor direction B
L298N Motor Driver	IN3	GPIO 21	Right Motor direction A
L298N Motor Driver	IN4	GPIO 22	Right Motor direction B
Relay Module 1	IN	GPIO 25	Crops Cutting Mechanism
Relay Module 2	IN	GPIO 26	Water Pump / Sprayer
Left IR Sensor	OUT / D0	GPIO 32	IR auto navigation input
Right IR Sensor	OUT / D0	GPIO 33	IR auto navigation input
Soil Moisture Sensor	AO (Analog Out)	GPIO 34	Analog input only (ADC1)
Rain Sensor	AO (Analog Out)	GPIO 35	Analog input only (ADC1)
HC-SR04 Ultrasonic	TRIG	GPIO 5	Output trigger
HC-SR04 Ultrasonic	ECHO	GPIO 23	Via 1k/2k voltage divider
DHT22	DATA	GPIO 27	10k pull-up to 3.3 V
16x2 I2C LCD (PCF8574T)	SDA	GPIO 16	Custom I2C bus
16x2 I2C LCD (PCF8574T)	SCL	GPIO 17	Custom I2C bus

Important Notes on GPIO Assignment: GPIO 34 and GPIO 35 are input-only pins on the ESP32 and are correctly used here for analog-only soil and rain sensor inputs. GPIO 21 and GPIO 22 are used by the motor driver (IN3 and IN4), which is

why the LCD I2C bus is reassigned to GPIO 16 (SDA) and GPIO 17 (SCL) to avoid I2C conflict. The HC-SR04 ECHO pin outputs 5 V; a voltage divider using R2 = 1 kΩ and R3 = 2 kΩ steps this down to approximately 3.33 V, protecting ESP32 GPIO 23. All sensor VCC connections use 3.3 V (preferred) to keep signals within the ESP32's safe input range. Relay module VCC uses 5 V, with a common GND shared between the relay module, ESP32, and motor driver.

3.3.1 Complete Circuit Diagram

Figure 3.2 shows the full circuit wiring diagram of AgriBot, including all sensor connections, motor driver wiring, relay module connections, LCD display I2C wiring, and the voltage divider for the HC-SR04 ECHO line.

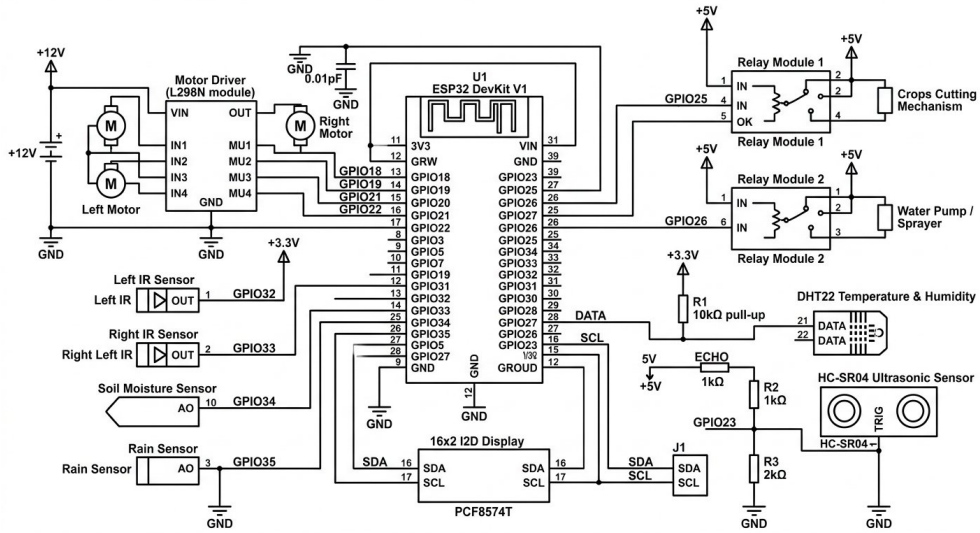


Figure 3.2: AgriBot Complete Circuit Diagram

As shown in Figure 3.2, the ESP32 DevKit V1 (U1) sits at the centre of the circuit. The L298N Motor Driver on the left receives GPIO 18, 19, 21, and 22 for motor control and connects to Left Motor and Right Motor via MU1–MU4 output terminals with +12V supply. Relay Module 1 (GPIO 25) and Relay Module 2 (GPIO 26) are on the upper right, each connected to +5V. The HC-SR04 ultrasonic sensor on the lower right connects TRIG to GPIO 5 and ECHO through a voltage divider (R2 = 1 kΩ, R3 = 2 kΩ) to GPIO 23. The DHT22 sensor connects its DATA pin to GPIO 27 with R1 = 10 kΩ pull-up to +3.3V. The 16x2 I2C LCD (PCF8574T) connects SDA to GPIO 16 and SCL to GPIO 17. Left IR, Right IR, Soil Moisture, and Rain sensors connect to GPIO 32, 33, 34, and 35 respectively. A 0.01 pF decoupling capacitor is placed on the ESP32 power supply line.

3.4 Step-by-Step Wiring

A common ground is first established across all modules: the ESP32 GND, motor driver GND, both relay module GNDs, all sensor GNDs, and the negative terminal of the 12 V motor battery are all connected together.

Motor Driver: The L298N module receives GPIO 18 to IN1, GPIO 19 to IN2, GPIO 21 to IN3, and GPIO 22 to IN4. The Left Motor connects to MU1/MU2 and the Right Motor to MU3/MU4. The 12 V motor battery connects to the VIN terminal.

Relay Modules: Relay 1 (Crops Cutting) IN pin connects to GPIO 25; Relay 2 (Water Pump/Sprayer) IN pin connects to GPIO 26. Both relay VCC pins use 5 V. The relays are active-LOW: the ESP32 drives the GPIO LOW to activate the relay.

IR Sensors: The Left IR sensor OUT pin connects to GPIO 32 and the Right IR sensor OUT pin connects to GPIO 33, both powered from 3.3 V with common ground.

Soil Moisture and Rain Sensors: The soil moisture AO connects to GPIO 34 and the rain sensor AO connects to GPIO 35, both powered from 3.3 V. The firmware reads the raw 12-bit ADC value (0–4095) and maps it to a 0–100% percentage using calibration constants.

HC-SR04 Ultrasonic Sensor: TRIG connects directly to GPIO 5. ECHO outputs 5 V and is stepped down using a voltage divider — R2 = 1 kΩ in series between ECHO and GPIO 23, and R3 = 2 kΩ between GPIO 23 and GND — producing approximately 3.33 V at GPIO 23. Sensor VCC connects to 5 V.

DHT22 Sensor: The DATA pin connects to GPIO 27 with a 10 kΩ pull-up resistor (R1) to 3.3 V. VCC connects to 3.3 V. The firmware enforces a minimum 2.5-second interval between readings.

16x2 I2C LCD: The PCF8574T expander (address 0x27) has SDA connected to GPIO 16 and SCL to GPIO 17, initialised in firmware via `Wire.begin(16, 17)` before `lcd.init()`. LCD VCC connects to 5 V.

3.5 Functional Requirements

ID	Requirement
FR1	The robot shall follow a crop row using two IR sensors (left and right) in Auto Mode.
FR2	The robot shall detect obstacles within range and stop if distance is at or below 15 cm.

FR3	The robot shall read soil moisture percentage and status (Dry/Normal/Wet) in real time.
FR4	The robot shall read rain sensor percentage and status (No Rain/Light Rain/Heavy Rain).
FR5	The robot shall read temperature and humidity from the DHT22 sensor.
FR6	The robot shall activate Relay 1 (Crops Cutting Mechanism) on dashboard command.
FR7	The robot shall activate Relay 2 (Water Pump/Sprayer) on dashboard command.
FR8	The robot shall connect as a WiFi Access Point and host a web dashboard at 192.168.4.1.
FR9	The farmer shall be able to switch between Auto Mode and Manual Mode via the dashboard.
FR10	The robot shall display sensor data and system status on a 16x2 I2C LCD.

3.6 Libraries Used

The following Arduino libraries are required and must be installed in the Arduino IDE: **WiFi.h** (ESP32 WiFi Access Point management, built-in), **WebServer.h** (HTTP server for the dashboard page), **WebSocketsServer.h** (WebSocket server, Markus Sattler), **ArduinoJson.h** (JSON serialisation for WebSocket messages), **Wire.h** (I2C communication on custom pins, built-in), **LiquidCrystal_I2C.h** (16x2 LCD control via I2C), **DHT.h** (DHT22 sensor reading, Adafruit DHT Sensor Library), and **Adafruit_Unified_Sensor.h** (dependency for the DHT library).

Chapter 4

Algorithm

This chapter presents the complete algorithmic logic of AgriBot, covering the main control loop, sensor reading procedures, auto navigation, relay control, LCD cycling, and WebSocket communication.

4.1 Main Control Algorithm

Listing 4.1: AgriBot Main Algorithm Pseudocode

```
START
INITIALIZE GPIO pins (motor, relay, IR, soil, rain, trig, echo, DHT22)
INITIALIZE I2C on GPIO 16 (SDA) and GPIO 17 (SCL)
INITIALIZE 16x2 LCD at address 0x27
INITIALIZE DHT22 on GPIO 27
INITIALIZE WiFi Access Point (SSID: "Smart Agriculture Robot")
START HTTP server on port 80
START WebSocket server on port 81

WHILE system is ON:

  // --- SENSOR READING (every 300ms) ---
  READ Left IR sensor (GPIO 32)   -> leftIRDetected
  READ Right IR sensor (GPIO 33)  -> rightIRDetected
  READ Soil Moisture (GPIO 34)    -> soilRawValue -> soilMoisturePercent
  READ Rain Sensor (GPIO 35)      -> rainRawValue -> rainPercent
  SEND 10us pulse from TRIG (GPIO 5)
  READ ECHO pulse on GPIO 23      -> distanceCm
  READ DHT22 on GPIO 27           -> temperatureC, humidityPercent

  // --- OBSTACLE DETECTION (in Auto Mode) ---
  IF distanceCm > 0 AND distanceCm <= 15 cm:
    STOP all motors
    motorStatus = "Obstacle Stopped"

  // --- AUTO NAVIGATION LOGIC ---
  ELSE IF autoMode is ON:
    IF leftIR NOT detected AND rightIR NOT detected:
      STOP motors
    ELSE IF leftIR detected AND rightIR detected:
      MOVE FORWARD (IN1=HIGH, IN2=LOW, IN3=LOW, IN4=HIGH)
    ELSE IF rightIR detected AND leftIR NOT detected:
      TURN RIGHT (IN1=HIGH, IN2=LOW, IN3=HIGH, IN4=LOW)
    ELSE IF leftIR detected AND rightIR NOT detected:
```

```
      TURN LEFT  (IN1=LOW, IN2=HIGH, IN3=LOW, IN4=HIGH)

  // --- RELAY CONTROL (from WebSocket command) ---
  IF relay1Command received:
    SET GPIO 25 LOW (relay ON) or HIGH (relay OFF)
  IF relay2Command received:
    SET GPIO 26 LOW (relay ON) or HIGH (relay OFF)

  // --- LCD DISPLAY (cycle every 2500ms) ---
  PAGE 0: Temp + Humidity
  PAGE 1: Rain Status + Object Distance
  PAGE 2: Soil Moisture % + Status
  PAGE 3: Mode (AUTO/MANUAL) + Motor Status
  PAGE 4: Left IR + Right IR state
  PAGE 5: Relay 1 (Cut) + Relay 2 (Spray) state

  // --- DASHBOARD BROADCAST (every 1000ms) ---
  SEND JSON via WebSocket to all clients:
    {motor, autoMode, relay1, relay2, leftIR, rightIR,
     soilRaw, soilPercent, soilStatus, rainRaw, rainPercent,
     rainStatus, distanceCm, ultrasonicStatus,
     temperatureC, humidity, dhtStatus}

END WHILE
```

4.2 Motor Control Functions

Four direction functions are implemented. Each function sets the four motor driver GPIO pins (18, 19, 21, 22) to HIGH/LOW combinations to produce the desired wheel motion.

Listing 4.2: Motor Control Function Implementation

```
void moveForward() {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);   // Left Motor forward
  digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);  // Right Motor forward
  motorStatus = "Moving Forward";
}
void turnLeft() {
  digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);  // Left Motor backward
  digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);  // Right Motor forward
  motorStatus = "Turning Left";
}
void turnRight() {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);   // Left Motor forward
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);   // Right Motor backward
  motorStatus = "Turning Right";
}
void stopCar() {
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
  motorStatus = "Stopped";
}
```

4.3 Auto Mode Navigation Logic

Listing 4.3: Auto Mode Navigation Logic

```
void autoModeControl() {
  if (!autoMode) return;
  // Obstacle check takes priority over IR navigation
  if (distanceCm > 0 && distanceCm <= OBSTACLE_STOP_DISTANCE_CM) {
    stopCar();
    motorStatus = "Obstacle Stopped";
    return;
  }
  // IR-based navigation
  if (!leftIRDetected && !rightIRDetected) stopCar();
  else if (leftIRDetected && rightIRDetected)  moveForward();
  else if (!leftIRDetected && rightIRDetected) turnRight();
  else if (leftIRDetected && !rightIRDetected) turnLeft();
}
```

4.4 Sensor Reading Algorithms

4.4.1 IR Sensor Reading

Listing 4.4: IR Sensor Reading

```
void readIRSensors() {
  leftIRDetected  = digitalRead(LEFT_IR_PIN)  == IR_DETECTED_STATE; // GPIO 32
  rightIRDetected = digitalRead(RIGHT_IR_PIN) == IR_DETECTED_STATE; // GPIO 33
}
```

4.4.2 Soil Moisture and Rain Sensor Reading

The raw ADC value from GPIO 34 (soil) and GPIO 35 (rain) is read using 12-bit resolution (0–4095). The value is mapped to a 0–100% scale using calibration constants:

- Soil: `SOIL_DRY_VALUE = 3500`, `SOIL_WET_VALUE = 1300`. Values below 30% are "Dry", 30–70% are "Normal", above 70% are "Wet".
- Rain: `RAIN_DRY_VALUE = 4095`, `RAIN_WET_VALUE = 1500`. Values below 20% are "No Rain", 20–60% are "Light Rain", above 60% are "Heavy Rain".

4.4.3 Ultrasonic Distance Measurement

A 10 μs HIGH pulse is sent to GPIO 5 (TRIG). The firmware reads the pulse duration on GPIO 23 (ECHO) using `pulseIn()` with a 20,000 μs timeout. Distance is calculated as:

$$\text{distanceCm} = \text{duration} \times 0.0343/2.0$$

A timeout returns -1 (Out of Range).

4.4.4 DHT22 Reading

The DHT22 is read no more than once every 2.5 seconds to comply with its sampling rate. Temperature (in °C) and humidity (in %) are read using `dht.readTemperature()` and `dht.readHumidity()`. If either value is NaN, the status is set to "DHT Error".

4.5 LCD Display Cycling Algorithm

The LCD cycles through 6 pages every 2.5 seconds:

1. Temperature (DHT22) and Humidity (DHT22)
2. Rain Status and Object Distance (ultrasonic)
3. Soil Moisture Percentage and Status
4. Operating Mode (AUTO/MANUAL) and Motor Status
5. Left IR and Right IR detection state
6. Relay 1 (Crops Cutting) and Relay 2 (Spray Water) state

4.6 WebSocket Communication Protocol

The ESP32 broadcasts a full status JSON to all connected dashboard clients every 1 second. The JSON payload includes all sensor values and system states:

Listing 4.5: JSON Status Broadcast

```
{
  "motor": "Moving␣Forward",
  "autoMode": true,
  "relay1": false,
  "relay2": false,
  "leftIR": true,
  "rightIR": true,
  "soilPercent": 45,
  "soilStatus": "Normal",
  "rainPercent": 10,
  "rainStatus": "No␣Rain",
  "distanceCm": 32.5,
  "ultrasonicStatus": "Clear",
  "temperatureC": 28.4,
  "humidity": 72,
  "dhtStatus": "OK"
}
```

Clients send JSON commands for motor direction, mode toggle, and relay control. On WebSocket disconnect, the robot stops if in Manual Mode.

4.7 Safety Implementation

- Relay modules are active-LOW. A software or power failure that leaves GPIO outputs floating HIGH keeps both relays OFF, preventing unintended tool or pump activation.
- The ultrasonic obstacle stop distance is set to 15 cm, giving the robot sufficient stopping margin.
- GPIO 34 and GPIO 35 are input-only pins used only for ADC reading, avoiding any risk of output configuration.
- The voltage divider on the ECHO line ensures the 5 V signal from HC-SR04 is safely reduced before reaching the ESP32.

Chapter 5

Discussion

This chapter presents the testing methodology, module test results, integrated system evaluation, and a discussion of AgriBot's performance and limitations.

5.1 Testing Methodology

Each module was first tested individually, then as part of the integrated system. Testing was performed in a controlled environment simulating agricultural conditions. The WiFi dashboard connection, motor movement, relay activation, and sensor accuracy were all verified systematically.

5.2 Module Test Results

5.2.1 WiFi and Dashboard Test

Connecting a smartphone to the SSID *Smart Agriculture Robot* (password: 12345678) and navigating to 192.168.4.1 successfully loaded the dashboard. The WebSocket connection was established within 1–2 seconds, and the connection status indicator turned green. The dashboard refreshed sensor data in real time at approximately 1-second intervals.

5.2.2 Motor Movement Test

Table 5.1: Motor Movement Test Results

Dashboard Button	**Expected Behaviour**	**Result**
Forward	Robot moves forward	Pass
Backward	Robot moves backward	Pass
Left	Robot turns left	Pass
Right	Robot turns right	Pass
Stop	Robot stops	Pass

Note: The motor direction logic was corrected from an earlier version where forward and backward were swapped. The current firmware uses the correct HIGH/LOW assignments.

5.2.3 Relay Test

Table 5.2: Relay Module Test Results

Test	**Expected Result**	**Result**
Crops Cutting button ON	Relay 1 activates (GPIO 25 LOW)	Pass
Crops Cutting button OFF	Relay 1 deactivates (GPIO 25 HIGH)	Pass
Spray Water button ON	Relay 2 activates (GPIO 26 LOW)	Pass
Spray Water button OFF	Relay 2 deactivates (GPIO 26 HIGH)	Pass

5.2.4 Sensor Test Results

Table 5.3: Sensor Module Test Results

Sensor	**Expected Dashboard Display**	**Result**
DHT22	Temperature in °C and Humidity in %	Pass
Soil Moisture	0–100% and Dry/Normal/Wet status	Pass
Rain Sensor	0–100% and No Rain/Light Rain/Heavy Rain	Pass
HC-SR04	Distance in cm and Clear/Obstacle Close status	Pass
Left IR	DETECTED / NOT DETECTED on dashboard	Pass
Right IR	DETECTED / NOT DETECTED on dashboard	Pass
16x2 LCD	Cycles all 6 pages every 2.5 seconds	Pass

5.2.5 Auto Mode Navigation Test

Table 5.4: Auto Mode Navigation Logic Test Results

IR Sensor State	**Expected Robot Action**	**Result**
Neither detected	Stop	Pass
Both detected	Move Forward	Pass
Right only detected	Turn Right	Pass
Left only detected	Turn Left	Pass
Obstacle $\leq$ 15 cm	Stop (Obstacle Stopped)	Pass

5.3 Integrated System Evaluation

The integrated system was tested end-to-end with all sensors, motors, relays, LCD, and dashboard active simultaneously. The system demonstrated stable WebSocket connections with no message loss under normal operating conditions. Sensor readings updated on both the LCD and the dashboard consistently. The auto mode correctly prioritised obstacle detection over IR navigation logic.

5.4 Discussion of Results

The two-IR-sensor auto navigation proved reliable for straight and gently curved crop rows, with sensor state transitions correctly triggering the four motion states and obstacle priority correctly enforced in all test scenarios.

The DHT22 sensor gave stable temperature and humidity readings, the soil moisture and rain sensors responded correctly to simulated wet/dry conditions, and the HC-SR04 ultrasonic sensor measured distances within ± 1 cm of a reference ruler up to 200 cm.

The WiFi dashboard loaded reliably on both Android and iOS smartphones, with WebSocket latency under 200 ms for a single client and immediate relay response to dashboard commands.

Limitations Observed: the DHT22's 2.5-second minimum sampling interval caused slight display delays during rapid environmental changes; IR navigation degraded on highly reflective or non-uniform outdoor terrain; the WiFi AP range was effective up to approximately 25 m in open environments; and no PWM speed control was implemented, so the robot operates at a single motor speed.

Chapter 6

Conclusion

6.1 Summary of the Work

This project designed and implemented AgriBot, an IoT-enabled autonomous agriculture robot built on the ESP32 DevKit V1 microcontroller. The robot integrates a two-IR-sensor auto navigation system, an HC-SR04 ultrasonic sensor for obstacle detection, an L298N motor driver for left and right DC motor control, two relay modules for a crops cutting mechanism and water sprayer, a DHT22 temperature and humidity sensor, an analog rain sensor, an analog soil moisture sensor, and a 16x2 I2C LCD display. A WiFi-based web dashboard provides real-time monitoring and remote control via WebSocket communication at 192.168.4.1. All objectives defined in Chapter 1 were successfully achieved.

6.2 Key Findings and Contributions

- AgriBot achieves reliable two-IR-sensor auto navigation with correct priority handling for obstacle detection over navigation logic.
- The GPIO conflict between the motor driver (GPIO 21/22) and the default I2C pins was resolved by assigning the LCD to a custom I2C bus on GPIO 16 and GPIO 17.
- The relay-based active-LOW safety architecture effectively prevents unintended crops cutting mechanism or pump activation on power loss.
- The ECHO voltage divider (1 kΩ / 2 kΩ) correctly protects the ESP32 from the 5 V HC-SR04 ECHO signal.
- The WiFi dashboard with WebSocket communication provides a practical real-time monitoring and control interface requiring only a smartphone and a browser.
- AgriBot demonstrates a full-featured agriculture robot at approximately 4,790 BDT, significantly more affordable than commercial alternatives.

6.3 Limitations of the Study

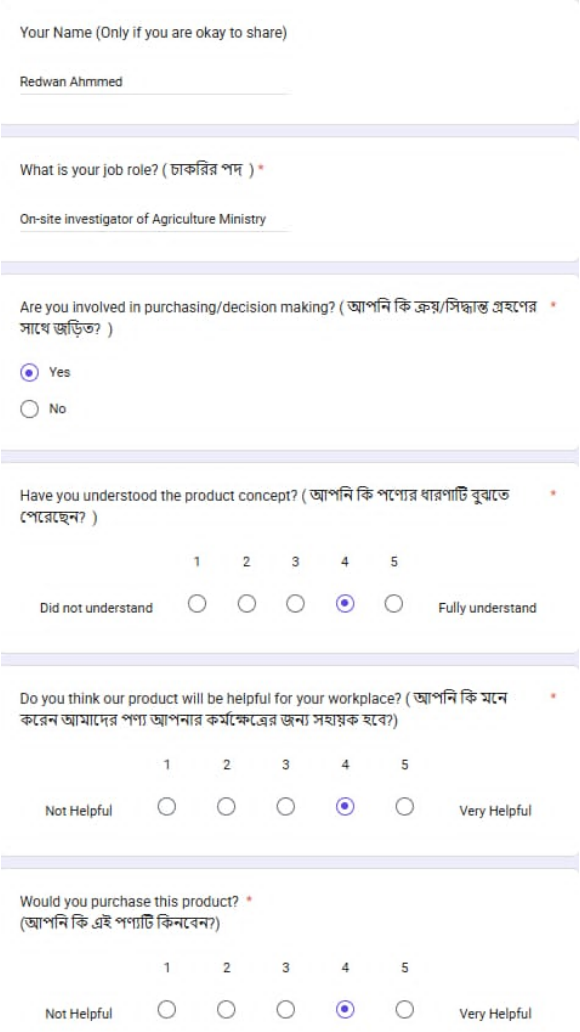
- GPIO 34 and GPIO 35 are input-only pins, limiting their use to analog input only. No PWM speed control is implemented for motors in the current firmware.
- DHT22's 2.5-second minimum update interval limits responsiveness to rapid environmental changes.
- IR-based navigation with only two sensors may degrade on complex or curved terrain.
- No persistent data logging for long-term field monitoring.
- The WiFi Access Point range is limited to approximately 20–30 m in open environments.

6.4 Recommendations and Future Work

- **PWM speed control:** Add ENA and ENB connections from the L298N to PWM-capable ESP32 pins for variable motor speed control.
- **Expand IR array:** Use 3–5 IR sensors for more precise curved row navigation.
- **Solar charging:** Integrate a solar panel and charging circuit to extend battery life.
- **GPS navigation:** Add GPS for large-field autonomous navigation without relying solely on IR line following.
- **Camera integration:** Use the ESP32-CAM module for live video streaming and crop disease detection.
- **Data logging:** Add an SD card or cloud connectivity for long-term sensor data logging and trend analysis.
- **Mobile app:** Develop a dedicated mobile application for easier farmer control.
- **Auto-irrigation logic:** Add firmware logic to automatically activate Relay 2 (spray) when soil moisture falls below a dry threshold and no rain is detected.

Appendix A

Survey 2


Your Name (Only if you are okay to share)

Redwan Ahmmed

What is your job role? (চাকরির পদ) *

On-site investigator of Agriculture Ministry

Are you involved in purchasing/decision making? (আপনি কি ক্রয়/সিদ্ধান্ত গ্রহণের সাথে জড়িত?) *

- (•) Yes
- ( ) No

Have you understood the product concept? (আপনি কি পণ্যের ধারণাটি বুঝতে পেরেছেন?) *

	1	2	3	4	5	
Did not understand	○	○	○	◉	○	Fully understand

Do you think our product will be helpful for your workplace? (আপনি কি মনে করেন আমাদের পণ্য আপনার কর্মক্ষেত্রের জন্য সহায়ক হবে?) *

	1	2	3	4	5	
Not Helpful	○	○	○	◉	○	Very Helpful

Would you purchase this product? *
(আপনি কি এই পণ্যটি কিনবেন?)

	1	2	3	4	5	
Not Helpful	○	○	○	◉	○	Very Helpful

Figure A.1: Survey form page 1

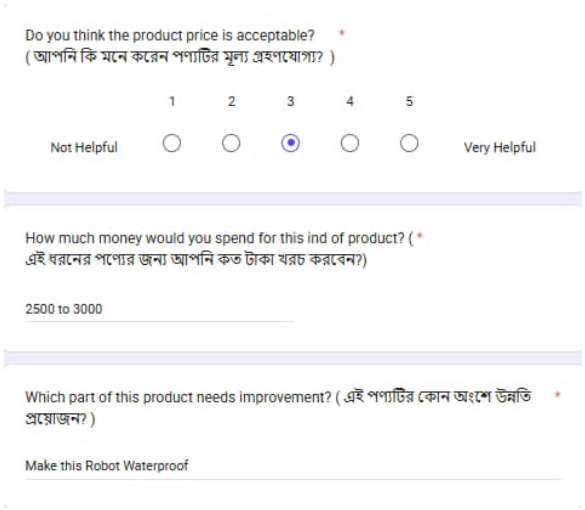


Figure A.2: Survey form page 2

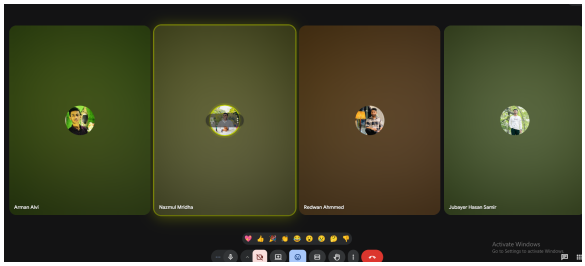


Figure A.3: Meeting

Appendix B

Team Contribution

Table B.1: Team Member Contribution

Name	**ID**	**Contribution**
Jubayer Hasan Samir	22201153	IR sensor wiring and line-following code; Ultrasonic sensor wiring and obstacle avoidance code; Blade motor L298N wiring and control code; Report writing
Nazmul Mridha	22201161	Power system design (battery, buck converter, relay); Wheel motor wiring and movement control; WiFi dashboard development; Report writing
Arman Alvi	22201164	Environment sensors (DHT11, Rain, Soil) wiring and logic; LCD and OLED display integration; Report writing and documentation

References

[1] M. A. Hossain, K. Ahmed, and T. Islam. Rice field navigation robot using ir sensor array. In *Proc. International Conference on Robotics and Automation (ICRA)*, pages 234–239, 2020.

[2] M. R. Islam and M. A. Hossain. Design and implementation of a line following robot for agricultural applications. *International Journal of Engineering and Technology*, 9(3):112–118, 2020.

[3] A. Khanna and S. Kaur. Evolution of internet of things (iot) and its significant impact in the field of precision agriculture. *Computers and Electronics in Agriculture*, 157:218–231, 2019.

[4] R. Kumar, P. Singh, and A. Sharma. Iot-based automated irrigation system using soil moisture sensor. *Journal of Agricultural Engineering*, 58(2):45–53, 2021.

[5] S. Patil, R. Kulkarni, and N. Desai. Smart farm monitoring system using esp8266 and iot. *International Journal of Advanced Research in Computer Science*, 13(1):78–84, 2022.

[6] Y. Zhang, L. Wang, and H. Chen. Design of an autonomous paddy harvesting robot with gps navigation. *Biosystems Engineering*, 185:51–63, 2019.